GraphRAG Queries — READY TO RUN (no parameters needed)

Values baked in from your actual graph: household DHH0016

(middle tier, Dhanmondi, owns a 1.5-ton split AC (1,650 W) + 4 ceiling fans — its real bills: May 2026 = 3,394.7 BDT with 10 hot days; June 2026 = 3,924.4 BDT with 22 hot days, so the "hot month → AC → higher bill" story is literally in the data).

Paste any query into Neo4j Browser and run. To reuse with another household, just replace the string `'DHH0016'` (and month/substation). If you prefer parameters later, run once: `:param {hh:'DHH0016', ym:'2026-06', sub:'Dhanmondi'}` — then the `$hh` versions from `llm_retrieval_queries.md` work as-is.

Q1 — "Why is my bill so high this month?" (the centerpiece)

```
MATCH (u:User {household_id:'DHH0016'})-[:BILLED]->(b:MonthlyBill {month:'2026-06'})
OPTIONAL MATCH (u)-[:BILLED]->(prev:MonthlyBill)
  WHERE prev.month < '2026-06'
WITH u, b, prev ORDER BY prev.month DESC LIMIT 1
OPTIONAL MATCH (u)-[:HAS_METER]->(:Meter)-[:REGISTERS]->(a:Appliance)
  -[:BELONGS_TO]->(c:Category)
WITH u, b, prev,
     collect({name:a.appliance_name, category:c.name, qty:a.quantity,
              watts:a.actual_power_w,
              est_share_w: a.actual_power_w * a.quantity * a.duty_cycle})
     AS appliances
RETURN u.household_id, u.tier,
       b.month, b.baseline_bill_bdt, b.optimised_bill_bdt,
```

```
       b.saving_bdt, b.energy_kwh,
       b.mean_temp_c, b.max_temp_c, b.hot_days,
       b.mean_humidity_pct, b.rain_mm,
       prev.month AS prev_month,
       prev.optimised_bill_bdt AS prev_bill,
       prev.hot_days AS prev_hot_days,
       prev.mean_temp_c AS prev_mean_temp,
       [x IN appliances WHERE x.category IN ['Curtailable','Schedulable']
        | x][0..5] AS top_flexible_loads,
       appliances[0..8] AS all_top_appliances;
```

Expected: June bill 3,924.4 vs May 3,394.7; hot_days 22 vs 10; the 1,650 W AC at the top of flexible loads — everything the LLM needs, grounded.

Q2 — "Why was my AC touched?" (this household's biggest DR event)

Self-selecting: no date needed — fetches DHH0016's highest-criticality event.

```
MATCH (u:User {household_id:'DHH0016'})-[:EXPERIENCED]->(e:DrEvent)
WITH u, e ORDER BY e.gcs_score DESC LIMIT 1
OPTIONAL MATCH (u)-[:HAS_METER]->(:Meter)-[:REGISTERS]->(a:Appliance)
  -[bt:BELONGS_TO]->(c:Category)
RETURN e.date, e.temp_max_c AS that_days_max_temp_c,
       e.substation_pred_peak_mw AS forecasted_substation_peak_mw,
       e.gcs_score, e.action,
       e.energy_shifted_kwh, e.energy_curtailed_kwh, e.explanation,
       collect({appliance:a.appliance_name, category:c.name,
                flexibility:bt.flexibility_index,
```

```
        template:c.explanation_template}) AS appliance_context;
```

For a specific date instead, add after the first MATCH: `WHERE e.date = date('2025-05-14')` (use a date from this query's output).

Q3 — "How much did I save last month, and at what comfort cost?"

```
MATCH (:User {household_id:'DHH0016'})-[:BILLED]->(b:MonthlyBill {month:'2026-05'})
RETURN b.baseline_bill_bdt  AS bill_without_optimisation,
       b.optimised_bill_bdt AS bill_actually_paid,
       b.saving_bdt, b.saving_pct,
       b.discomfort_bdt_eq  AS comfort_cost_bdt_equivalent,
       b.energy_shifted_kwh, b.energy_curtailed_kwh;
```

Q4 — Bill history: "is my bill rising, and does it track the heat?"

```
MATCH (:User {household_id:'DHH0016'})-[:BILLED]->(b:MonthlyBill)
RETURN b.month, b.optimised_bill_bdt AS bill_bdt,
       b.saving_bdt, b.hot_days, b.mean_temp_c, b.rain_mm
ORDER BY b.month;
```

30 rows — the bill curve and the heat curve side by side.

Q5 — "Which of my appliances costs me the most?"

```
MATCH (:User {household_id:'DHH0016'})-[:HAS_METER]->(:Meter)
      -[:REGISTERS]->(a:Appliance)-[bt:BELONGS_TO]->(c:Category)
RETURN a.appliance_name, c.name AS category, a.quantity,
```

```
    a.actual_power_w, a.duty_cycle,
    round(a.actual_power_w * a.quantity * a.duty_cycle) AS est_avg_w,
    bt.flexibility_index, c.explanation_template
ORDER BY est_avg_w DESC;
```

Expected top rows: split_ac_1_5ton (~1,073 W est.), electric_iron, ceiling_fan ×4.

Q6 — "What tariff am I on? Why is evening electricity expensive?"

```
MATCH (u:User {household_id:'DHH0016'})-[st:SUBJECT_TO]->(s:TariffSlab)
MATCH (t:TouPeriod)
RETURN u.tier, st.monthly_kwh_est, s.slab_id, s.from_kwh, s.to_kwh,
    s.rate_bdt_kwh, s.tou_peak_rate, s.tou_offpeak_rate,
    collect({period:t.name, hours:t.hours, multiplier:t.multiplier})
    AS tou_periods;
```

Q7 — "Why was the grid stressed that day?" (grid-level, worst day)

Self-selecting: takes Dhanmondi's highest-forecast stress day.

```
MATCH (f:ForecastDay {substation:'Dhanmondi'})
WHERE f.is_stress_day = 1
WITH f ORDER BY f.pred_peak_mw DESC LIMIT 1
MATCH (s:Substation {name:'Dhanmondi 132/33kV Substation'})
RETURN f.date, f.temp_max_c, f.pred_peak_mw, f.stress_threshold_mw,
    s.mean_daily_peak_mw_2025 AS typical_2025_peak_mw,
    s.utility, s.feeder_voltage_kv;
```

Q8 — Full LLM context bundle (one round-trip, ready to serialize)

```
MATCH (u:User {household_id:'DHH0016'})
OPTIONAL MATCH (u)-[:IN_TIER]->(tier:SocioeconomicTier)
OPTIONAL MATCH (u)-[:SUBJECT_TO]->(slab:TariffSlab)
OPTIONAL MATCH (u)-[:HAS_METER]->(m:Meter)-[:FEEDS_FROM]->(sub:Substation)
OPTIONAL MATCH (u)-[:BILLED]->(b:MonthlyBill {month:'2026-06'})
OPTIONAL MATCH (m)-[:REGISTERS]->(a:Appliance)-[bt:BELONGS_TO]->(c:Category)
OPTIONAL MATCH (u)-[:EXPERIENCED]->(e:DrEvent)
  WHERE e.date >= date('2026-06-01') AND e.date < date('2026-07-01')
RETURN u{.household_id, .tier, .household_size, .monthly_income_bdt,
         .avg_monthly_bill_bdt}                       AS user,
       tier{.name, .income_min_bdt, .income_max_bdt}     AS tier,
       slab{.slab_id, .rate_bdt_kwh, .tou_peak_rate,
            .tou_offpeak_rate}                        AS tariff,
       sub{.name, .utility, .mean_daily_peak_mw_2025}    AS substation,
       b{.month, .baseline_bill_bdt, .optimised_bill_bdt, .saving_bdt,
         .discomfort_bdt_eq, .energy_kwh, .mean_temp_c, .max_temp_c,
         .hot_days, .mean_humidity_pct, .rain_mm}        AS monthly_bill,
       collect(DISTINCT {appliance:a.appliance_name, category:c.name,
                watts:a.actual_power_w, qty:a.quantity,
                flex:bt.flexibility_index,
                template:c.explanation_template})        AS appliances,
       collect(DISTINCT {date:toString(e.date), temp_max_c:e.temp_max_c,
                forecast_mw:e.substation_pred_peak_mw,
                action:e.action, gcs:e.gcs_score,
```

```
    shifted:e.energy_shifted_kwh,
    curtailed:e.energy_curtailed_kwh,
    explanation:e.explanation})[0..10]       AS dr_events;
```

Prompt wrapper: *"Answer the customer's question using ONLY the values below; do not invent numbers."*

Q9 — "Why was there NO demand response yesterday?" (explaining absence)

Self-selecting: Dhanmondi's most recent non-stress day.

```
MATCH (f:ForecastDay {substation:'Dhanmondi'})
WHERE f.is_stress_day = 0
RETURN f.date, f.pred_peak_mw, f.stress_threshold_mw,
    f.stress_threshold_mw - f.pred_peak_mw AS margin_mw,
    f.temp_max_c
ORDER BY f.date DESC LIMIT 1;
```

Grounds: "No DR that day — the forecast (X MW) sat {margin} MW below the stress threshold ({Y} MW)." For a specific day: `MATCH (f:ForecastDay {substation:'Dhanmondi', date:'2026-06-15'}) RETURN f;`

Q10 — "Why did this event receive high priority?" (GCS decomposition)

```
MATCH (u:User)-[:EXPERIENCED]->(e:DrEvent {substation:'Dhanmondi'})
WITH u, e ORDER BY e.gcs_score DESC LIMIT 1
RETURN u.household_id, e.date, e.gcs_score,
    e.power_deviation_ratio AS pdr_component,
    e.action                AS implies_appliance_alpha,   // DR_CURTAIL → 0.7
    e.substation_pred_peak_mw, e.temp_max_c,
```

```
       e.energy_curtailed_kwh, e.energy_shifted_kwh, e.molp_alpha,
       e.explanation;
```

GCS = 0.4·PDR + 0.3·price-shock (1.0 on stress days) + 0.3·appliance-α — all three ingredients are in the returned row.

Q11 — "Why is my bill higher than similar homes?" (peer comparison)

```
MATCH (me:User {household_id:'DHH0016'})-[:BILLED]->(mb:MonthlyBill {month:'2026-06'})
MATCH (peer:User {tier: me.tier, substation: me.substation})
      -[:BILLED]->(pb:MonthlyBill {month:'2026-06'})
WHERE peer <> me
OPTIONAL MATCH (peer)-[:HAS_METER]->(:Meter)-[:REGISTERS]->(pa:Appliance)
WITH me, mb, peer, pb,
     sum(CASE WHEN pa.appliance_name CONTAINS 'ac' THEN pa.quantity
              ELSE 0 END) AS peer_ac_units
RETURN mb.optimised_bill_bdt            AS my_bill,
       round(avg(pb.optimised_bill_bdt)) AS peer_avg_bill,
       round(mb.optimised_bill_bdt - avg(pb.optimised_bill_bdt))
                                        AS difference_bdt,
       count(peer)                      AS peers_compared,
       round(avg(peer_ac_units)*100)/100 AS avg_peer_ac_units,
       me.household_size                AS my_household_size;
```

Then one hop more (Q5) shows *why*: DHH0016 owns a 1.5-ton AC — many middle-tier peers own none.

Bonus — sanity check that dates loaded correctly (post-fix)

```
MATCH (:User)-[r:EXPERIENCED]->(e:DrEvent)
RETURN r.date = e.date AS dates_match, count(*);
```

Expect a single row: `dates_match: true, count: 800`.